

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	SHAFFRIA.A	Reg. No.:	192571047
Section / Batch:	B TECH CSBS	Date:	04.08.2026
Instructions to Students • Answer the following question. Total Marks: 100. • Write the algorithm and execute the code for the given experiment. • Follow a well-structured, systematic approach to design and implement an optimal solution. • Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.			

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I SHAFFRIA.A (192571047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CODING AND OUTPUT:

CO3 AT1-EXPERIMENTS

[← Back](#)

Language: Python C C++ Java

QUESTIONS

Q1

Quick Sort

100 marks

1/1 answered

Q1 Quick Sort

100 marks

Evaluate Quick Sort performance by implementing tail recursion optimization. Compare execution time with standard recursive implementation. Analyze stack usage and memory efficiency. Interpret the results and justify improvements achieved through optimization.

Your Code

```
1 import random,time
2 def qsort_rec(a):
3     if len(a)<=1:return a
4     p=a[len(a)//2]
5     return qsort_rec([x for x in a if x<p])+[x for x in a if x==p]+qsort_rec([x for x in a if x>p])
6
7 def qsort_tail(a):
8     st=[(0,len(a)-1)]
9     while st:
10         l,r=st.pop()
11         if l>=r: continue
12         p=a[(l+r)//2]; i,j=l,r
13         while i<=j:
14             while a[i]<p: i+=1
15             while a[j]>p: j-=1
16             if i<=j: a[i],a[j]=a[j],a[i]; i+=1; j-=1
```

Input

stdin input

Output

Recursive: 0.1718 s
Tail-opt: 0.125 s

1/1 answered

Your Code

```

1 import random,time
2 def qsort_rec(a):
3     if len(a)<=1: return a
4     p=a[len(a)//2]
5     return qsort_rec([x for x in a if x<p])+[x for x in a if x==p]+qsort_rec([x for x in a if x>p])
6
7 def qsort_tail(a):
8     st=[(0,len(a)-1)]
9     while st:
10         l,r=st.pop()
11         if l>=r: continue
12         p=a[(l+r)//2]; i,j=l,r
13         while i<=j:
14             while a[i]<p: i+=1
15             while a[j]>p: j-=1
16             if i<=j: a[i],a[j]=a[j],a[i]; i+=1; j-=1
17         if l<j: st.append((l,j))
18         if i<r: st.append((i,r))
19     return a
20 arr=[random.randint(0,1e6) for _ in range(50000)]
21 t=time.time(); qsort_rec(arr[:]); print("Recursive:",round(time.time()-t,3))
22 t=time.time(); qsort_tail(arr[:]); print("Tail-opt:",round(time.time()-t,3))

```

Input

stdin input

Output

Recursive: 0.1718 s
Tail-opt: 0.125 s

Submitted

Standard Recursive QuickSort (qsort_rec):

- Works by **dividing** the list into three parts:
 - Elements smaller than the pivot
 - Elements equal to the pivot
 - Elements greater than the pivot
- Then recursively sorts the smaller and greater parts.

Drawback:

- Creates many temporary lists (left, mid, right) → extra memory usage.
- Deep recursion → risk of hitting Python's recursion limit and higher stack usage.

Tail-Optimized QuickSort (qsort_tail):

- Instead of recursion, it uses an **explicit stack list** (st) to track subarray ranges.
- Sorting happens **in-place** by swapping elements around the pivot.

Advantages:

- Avoids recursion depth issues → no stack overflow.
- More memory-efficient since it doesn't build new lists repeatedly.
- Faster for large arrays because it reduces overhead from function calls.

Performance Comparison:

- Both functions are tested on a large random array.
- **Recursive version:** Simple but slower and memory-heavy.

- **Tail-optimized version:** Faster, safer for big inputs, and more efficient in memory usage.

Conclusion:

Tail recursion optimization in Python is essentially **turning recursion into iteration.**

- It improves **execution time** by reducing function call overhead.
 - It improves **memory efficiency** by avoiding deep recursion and extra list allocations.
 - The optimized version is more **scalable** for large datasets.
-